



МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ ПЕДАГОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»

**ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И
ТЕХНОЛОГИЧЕСКОГО ОБРАЗОВАНИЯ**
Кафедра информационных технологий и электронного обучения

Основная профессиональная образовательная программа
Направление подготовки 09.03.01 Информатика и вычислительная техника
Направленность (профиль) «Технологии разработки программного обеспечения»
форма обучения – очная

Курсовая работа

«Реализация сайта-портфолио IT-специалиста
на статическом генераторе Lektor»

Обучающегося 4 курса
Суркова Андрея Анатольевича

Научный руководитель:
кандидат физико-математических наук,
доцент кафедры ИТиЭО
Власов Дмитрий Викторович

Санкт-Петербург
2025

ОГЛАВЛЕНИЕ

Введение.....	3
Глава 1. Требования и инструменты	4
1.1. Формирование требований.....	4
1.2. Анализ инструментов разработки	4
1.3. Обоснования выбора CMS Lektor	5
1.4. Выбор платформы для размещения сайта	6
1.5. Выводы по главе.....	7
Глава 2. Проектирование и реализация сайта-портфолио.	8
2.1. Создание сайта при помощи Lektor	8
2.2. Структура сайта и навигация	8
2.3. Создание и наполнение страниц.....	9
2.4. Публикация веб-портфолио	11
2.5. Выводы по главе.....	12
Заключение	13
Список источников	14
Приложение 1.....	15

Введение

В современном мире остро стоит вопрос высокой конкуренции на рынке труда. Персональное веб-портфолио может послужить одновременно эффективным и экономичным инструментом, помогающим в решении этой проблемы. Оно позволяет не только демонстрировать потенциальному работодателю практические навыки соискателя, его знания и готовые проекты, но и систематизировать результаты деятельности, что обеспечивает автору возможность объективного отслеживания и анализа собственного профессионального роста [1, 2, 3]. Настоящая работа включает в себя обзор некоторых инструментов для реализации портфолио, описание процесса реализации портфолио, анализ и синтез литературных источников, в чем и заключается актуальность данной работы.

Цель работы – разработка и внедрение функционального сайта-портфолио программиста с использованием статического генератора контента Lektor.

Для достижения цели было необходимо решить следующие задачи:

1. Проанализировать современные требования к сайтам-портфолио программистов и изучить возможности CMS Lektor.
2. Спроектировать структуру, пользовательский интерфейс и контент-модель будущего сайта.
3. Разработать сайт-портфолио.
4. Выполнить развертывание сайта на публичный хостинг.

Объект исследования – процесс разработки веб-портфолио.

Предмет исследования – использование CMS Lektor для создания портфолио программиста.

Настоящая курсовая работа включает введение, 2 главы и заключение; содержит 15 страниц; 2 таблицы; 4 рисунка; 1 приложение; список источников, содержащий 10 источников.

Глава 1. Требования и инструменты

1.1. Формирование требований

Разработка любого программного продукта, включая сайт-портфолио, начинается с чёткого определения его целей и границ. Основная цель данного проекта – создать эффективный инструмент для демонстрации профессиональных компетенций разработчика и проектного опыта целевой аудитории: техническим рекрутерам, руководителям IT-отделов и потенциальным заказчикам.

Для достижения этой цели были проанализированы существующие сайты-портфолио, а также шаблоны и обзорные статьи существующих веб-портфолио [4, 5, 6]. Исходя из проведенного анализа, были сформированы представленные ниже требования.

Сайт должен обладать минималистичным дизайном с простой навигацией, так как визуальное оформление сайта должно концентрировать внимание пользователя на содержании, а не на себе.

Интерфейс сайта должен быть читаемым как на персональных компьютерах, так и на телефонах.

Структура сайта должна обладать выраженной иерархической организацией для группирования тематических материалов. Это решение облегчит поиск информации на сайте и упростит навигацию.

Разработанный сайт должен обладать высокими показателями производительности, чтобы иметь возможность развертывания на недорогих или бесплатных облачных платформах для размещения сайтов.

Таким образом, сформулированные требования образуют целостную систему, где минималистичный дизайн обеспечивает основу для адаптивности, иерархическая структура делает контент управляемым и доступным, а высокая производительность становится техническим фундаментом для экономичной и надёжной публикации проекта. Данный комплекс требований послужил основой для последующего сравнительного анализа технологических решений.

1.2. Анализ инструментов разработки

Для реализации сайта был проанализирован три основных вида конструкторов, а именно: динамические конструкторы, такие как WordPress или Joomla, онлайн-конструкторы (Tilda, Wix) и статические генераторы сайтов (Lektor CMS, Hugo CMS). Ниже приведена сравнительная таблица по ключевым критериям.

Таблица 1. Сравнительный анализ инструментов разработки

Критерий	Динамические CMS	Онлайн-конструкторы	Статические CMS
Производительность	Снижена из-за работы с базой данных и сервером.	Снижена из-за загрузки тяжелых клиентских скриптов.	Максимальная производительность из-за работы с готовым HTML файлом.
Безопасность	Средняя из-за плагинов и частых обновлений, базы данных и работы с сервером.	Высокая	Высокая благодаря отсутствию базы данных и серверного исполнения кода.
Контроль и гибкость	Высокая гибкость	Очень низкая гибкость	Высокая гибкость
Зависимость от сторонних платформ	Низкая, зависимость от хостинга.	Критически высокая.	Минимальная, легко заменяемый хостинг.
Сложность разработки	Высокая	Низкая	Средняя

Основываясь на информации, приведенной в таблице 1, было принято решение использовать статические генераторы, так как они соответствуют требованиям к разработке, предлагают оптимальный баланс между производительностью, безопасностью, контролем и удобством разработки.

1.3. Обоснования выбора CMS Lektor

В рамках задачи разработки персонального сайта-портфолио был проведен сравнительный анализ современных генераторов статических сайтов (Static Site Generators, SSG), включая такие популярные решения, как Jekyll и Hugo. По итогам анализа в качестве базового инструмента был выбран Lektor, что обусловлено совокупностью его технологических преимуществ и уникальных особенностей, наиболее полно соответствующих требованиям проекта [7].

В отличие от многих SSG, где структура данных зачастую произвольна, Lektor использует систему явно определенных моделей (Models). Разработчик заранее задает типы полей, таких как текст, Markdown, изображение, список или выбор из вариантов для каждого типа контента. Это превращает процесс наполнения сайта не в редактирование Markdown-файлов с необходимостью помнить синтаксис, а в интуитивное заполнение структурированных форм, гарантирующее целостность и однородность данных. Для портфолио программиста это означает четкие шаблоны для добавления новых проектов с обязательными полями.

Lektor поставляется с готовой веб-интерфейсной административной панелью. Это критически важно для долгосрочного сопровождения сайта: для внесения правок или добавления новой информации не требуется доступ к файловой системе или знание Git. Разработчик может обновлять контент через удобный браузерный интерфейс, что кардинально отличает Lektor от таких инструментов, как Hugo или Jekyll, и сближает его по удобству с более «тяжелыми» CMS, но без их недостатков.

Lektor предлагает исключительно простой цикл локальной разработки с автоматической перезагрузкой и встроенными инструментами публикации. Развертывание на бесплатный хостинг, такой как GitHub Pages, выполняется одной командой или через админ-панель. Эта интеграция минимизирует рутинные операции по сборке и загрузке файлов, позволяя сосредоточиться на содержании и дизайне.

Таким образом, выбор Lektor обусловлен не единичным преимуществом, а синергией его возможностей. Он представляет собой инструмент, который с одной стороны, уважает время и навыки программиста, предоставляя полный контроль и гибкость, а с другой — заботится о будущем проекта, предлагая профессиональные средства для управления контентом. Это делает его идеально подходящим для создания и долгосрочного сопровождения современного, быстрого и удобного сайта-портфолио.

1.4. Выбор платформы для размещения сайта

После обоснования выбора Lektor в качестве инструмента разработки, следующим критически важным решением стал выбор платформы для хостинга и публикации финального продукта. Для размещения статического сайта-портфолио была выбрана платформа GitHub Pages.

Данный выбор обусловлен комплексом технических, экономических и операционных преимуществ, которые напрямую соответствуют сформулированным ранее требованиям к проекту.

GitHub Pages предоставляет возможность хостинга статических сайтов без каких-либо финансовых затрат, что идеально соответствует требованию экономической эффективности и делает проект доступным для любого разработчика, включая студентов.

Платформа является неотъемлемой частью экосистемы GitHub, что упрощает размещение работ и проектов программиста на сайте. Помимо этого, Lektor имеет встроенную поддержку деплоя на GitHub Pages одной командой или через админ-панель, а инструмент «GitHub Actions» позволяет настроить конвейер непрерывной интеграции и доставки, при котором любое обновление кода в репозитории автоматически запускает сборку и публикацию обновленной версии сайта [7].

Статические файлы сайта обслуживаются через глобальную сеть доставки контента (CDN) GitHub [8]. Это обеспечивает высокую скорость загрузки для пользователей по всему миру и отказоустойчивость сайта, поддерживаемого крупнейшей платформой для разработчиков.

1.5. Выводы по главе

Проведенный в главе анализ позволил перейти к детальному техническому проекту. Были определены требования к сайту-портфолио. Проведенный анализ технологических подходов позволил определиться с инструментами разработки. В качестве основного инструмента была выбрана CMS Lektor, сочетающая гибкость, независимость от сторонних платформ, высокую производительность и удобство управления контентом.

Глава 2. Проектирование и реализация сайта-портфолио.

2.1. Создание сайта при помощи Lektor

Для создания сайта при помощи Lektor необходимо подготовить среду разработки и установить этот инструмент. Поскольку Lektor написан на языке программирования Python, в качестве интегрированной среды разработки была выбрана программа PyCharm. При помощи PyCharm было создано виртуальное окружение, в которое была установлена система управления зависимостями `pipx`, а также сама CMS Lektor. После установки Lektor была использована команда `lektor quickstart`, которая запускает процесс создания проекта. Во время создания проекта указывается название и путь проекта, имя автора [7]. Команды, использованные при установке Lektor, представлены ниже в таблице 2.

Таблица 2.

Команда	Краткое описание
<code>python3 -m pip install --user pipx</code>	Команда установки <code>pipx</code> - менеджера приложений, который является необходимым инструментом для установки lektor
<code>pipx install lektor</code>	Команда установки Lektor
<code>lektor quickstart</code>	Создает основу для проекта, инициализирует его.
<code>lektor server</code>	Запускает встроенный локальный сервер для разработки проекта, предоставляет адрес для предпросмотра сайта и управления через админ-панель.

2.2. Структура сайта и навигация

Основной принцип организации контента — вложенная иерархия, обеспечивающая строгую логическую связность и интуитивную навигацию. Пользователь начинает работу с главной страницы, откуда получает доступ к разделам, соответствующим семестрам обучения. Каждый семестр, в свою очередь, содержит перечень изучавшихся дисциплин. Выбрав конкретную дисциплину, пользователь попадает на страницу с работами, выполненными в её рамках. Для дисциплин, изучавшихся на протяжении нескольких семестров, предусмотрены ссылки на соответствующие подстраницы, что позволяет сохранить целостность материала. Помимо иерархического меню, быстрый доступ ко всем ключевым разделам (семестрам, а также странице «Об авторе») предоставляет фиксированная навигационная панель в верхней части сайта.

Указанная структура была эффективно реализована благодаря инструменту работы с шаблонами Jinja2, который является ядром Lektor и позволяет отделить логику

представления от содержимого. Все страницы сайта строятся по принципу наследования шаблонов.

Разработанный сайт-портфолио содержит базовый шаблон (layout.html), который определяет общий каркас всех страниц (HTML-структуру, подключение стилей и навигационную панель). Для написания шаблонов страниц были изучены необходимые материалы [7, 9, 10].

Дочерние шаблоны (например, шаблоны курса или дисциплины) наследуют этот каркас от базового шаблона и определяют уникальную область контента для своего типа страницы. Это позволяет создавать новые страницы (например, добавить дисциплину в 4-й семестр) без дублирования кода, лишь указав соответствующий шаблон.

2.3. Создание и наполнение страниц

Наполнение контента происходит не в HTML-файлах, а в отдельных файлах с расширением .lr, которые используют облегчённый Lektor File Format, интуитивно понятный благодаря сходству с Markdown. В этих файлах хранится чистое содержимое (текст, ссылки, метаданные). При сборке сайта система Lektor автоматически подставляет данные из файлов .lr в соответствующие места заранее подготовленных шаблонов. Такой подход — строгое разделение данных, их структуры и представления — обеспечивает простоту долгосрочного наполнения через админ-интерфейс и гарантирует единообразие дизайна на всех страницах сайта.

В Lektor структура URL-адресов страниц сайта напрямую и зеркально отражает файловую структуру папок и файлов .lr внутри проекта. Это ключевая концепция, обеспечивающая предсказуемость и контроль над навигацией. Так например, структура папок, изображенная на рисунке 1, размещает страницу с работами по программированию по адресу «/tretiy-semester/prog3/»

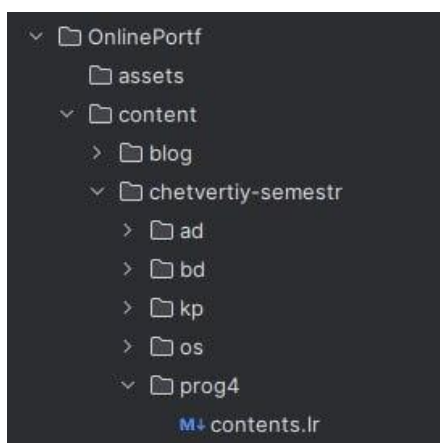


Рисунок 1. Пример структуры фрагмента сайта.

Можно отметить, что благодаря такому подходу, проектирование информационной архитектуры сайта сводится к логическому структурированию контента в файловой системе, что исключает ошибки в маршрутизации и облегчает масштабирование проекта.

Создание и наполнение страниц на сайте возможно как при помощи прямой работы с файлами, так и через административную панель сайта. Доступ к админ-панели сайта возможен с любой страницы, и осуществляется через нажатие на соответствующую кнопку (обозначенную на рисунке 2), расположенную в правом верхнем углу экрана.

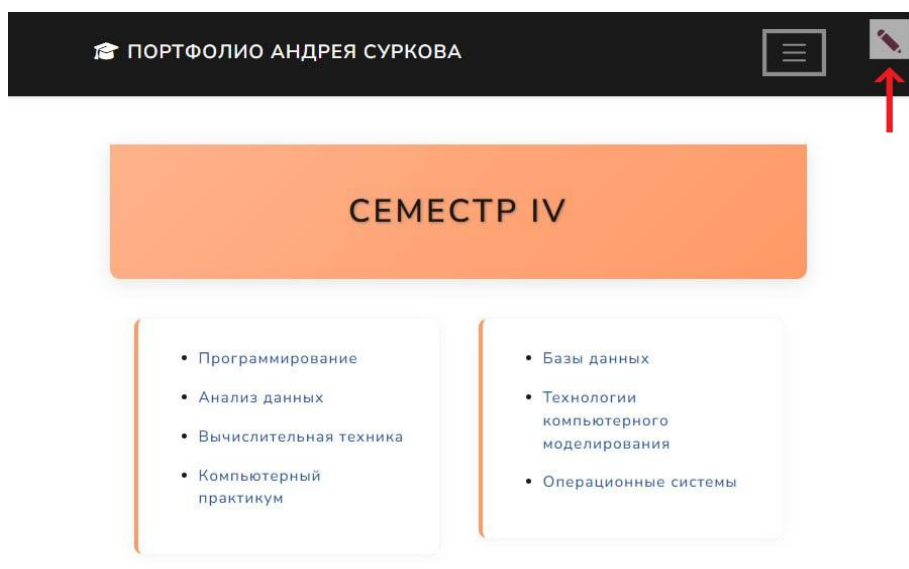


Рисунок 2. Расположение кнопки перехода в админ-панель.

Админ-панель Lektor — это его ключевое преимущество, превращающее статический генератор в полноценную систему управления контентом. Это веб-интерфейс, который позволяет создавать и редактировать страницы сайта, не требуя при этом правки исходных файлов вручную. Административная панель позволяет вносить изменения в тексты, изображения, метаданные и структуру страниц через удобные формы, даёт возможность создавать, удалять и перемещать страницы в древовидной структуре сайта напрямую из браузера. Все изменения в админ-панели можно сразу же увидеть, так как Lektor автоматически перестраивает изменённые страницы. Админ-панель также включает инструменты для работы с изображениями и предоставляет возможность напрямую публиковать готовый сайт на удалённые серверы, такие как GitHub Pages, без использования командной строки.

Админ панель для страницы дисциплин третьего семестра обучения представлена на рисунке 3.

Закрашенная часть изображения содержит персональный токен доступа, который позволяет аутентифицировать пользователя в GitHub.

После настройки конфигурации сервера кнопка публикации становится доступной в веб-интерфейсе административной панели. При её нажатии Lektor автоматически выполняет сборку текущей версии сайта и отправляет сгенерированные статические файлы в специальную ветку указанного удалённого репозитория на GitHub.

Вторым методом публикации сайта является метод публикации через командную строку. Этот метод осуществляется через использование команды «`lektor deploy ghpages`», которую необходимо ввести после настройки файла конфигурации. Данная команда является полным аналогом действия через админ-панель: она также инициирует сборку проекта и последующий деплой на предварительно настроенный хостинг. Главное преимущество этого подхода — возможность его автоматизации. Например, можно настроить сценарий в системе контроля версий Git так, чтобы команда `lektor deploy` выполнялась автоматически при каждом обновлении основной ветки репозитория с исходным кодом проекта, обеспечивая непрерывное обновление сайта.

Независимо от выбранного метода, результат идентичен — статические файлы сайта размещаются на бесплатном и высоконадёжном хостинге GitHub Pages. Lektor абстрагирует сложные операции с Git, предоставляя разработчику единый и простой интерфейс для публикации, что полностью соответствует цели создания сайта, простого в долгосрочном сопровождении.

2.5. Выводы по главе

В рамках второй главы был проведен полный цикл практической реализации сайта-портфолио программиста на базе статического генератора Lektor. Целью данной главы являлось проектирование, разработка и развертывание работоспособного цифрового продукта, соответствующего требованиям, сформулированным во введении.

Заключение

В рамках данной курсовой работы была достигнута поставленная цель — разработан и внедрен функциональный сайт-портфолио программиста с использованием статического генератора контента Lektor. Все сформулированные задачи успешно выполнены, что подтверждается созданием работоспособного, соответствующего современным стандартам цифрового продукта. В ходе исследования были изучены ключевые требования к эффективному сайту-портфолио, была спроектирована четкая иерархическая информационная архитектура, была реализована система шаблонов с наследованием и проведено наполнение контента через файлы формата .lr и админ-интерфейс. Завершающим этапом стала публикация сайта на платформе GitHub Pages. С итоговым сайтом-портфолио можно ознакомиться, перейдя по ссылке в приложении 1.

Список источников

1. Why Does Every Professional Business Need a Portfolio Website? // Acquaint Softtech URL: <https://acquaintsoft.com/blog/benefits-of-developing-portfolio-website> (дата обращения: 10.12.2025).
2. Lina Sierra, Piedad Martin, M del Carmen Sierra (2012). E-portfolio, a Self-assessment Tool for Evaluating Subject Content Acquisition: A Practical Example in a European Project. Journal of Modern Education Review January 2012, Volume 2, No. 1, pp. 56–61
URL: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2096937 (дата обращения: 10.12.2025).
3. Kate Dunne, Pauline Logue (2021). A Higher Education Action Research Study on the Effectiveness of an ePortfolio as a Learning Tool to Promote Reflective Professional Development. Irish Journal of Technology Enhanced Learning (2021) Volume 6, Issue 1 URL: <https://journal.ilta.ie/index.php/telji/article/view/79/106> (дата обращения: 10.12.2025).
4. Ахметвалеева Л. В., Хапаева Л. Х., Алиева М., В. Обзор сайта-портфолио для представления достижений и реализации проектов. Журнал прикладных исследований 2024, doi: 10.47576/2949-1878.2024.11.11.004 URL: <https://cyberleninka.ru/article/n/obzor-sayta-portfolio-dlya-predstavleniya-dostizheniy-i-realizatsii-proektov/viewer> (дата обращения: 16.12.2025).
5. Chase Ohlson Portfolio // Chase Ohlson URL: <https://chaseohlson.com> (дата обращения: 16.12.2025).
6. Lektor Themes // Andrew-Shay URL: <https://github.com/lektor/lektor-themes> (дата обращения: 20.11.2025).
7. Lektor Documentation // Getlektor URL: <https://www.getlektor.com/docs/> (дата обращения: 17.11.2025).
8. GitHub Pages documentation // GitHub Docs URL: <https://docs.github.com/en/pages> (дата обращения: 17.11.2025).
9. Your first website // MDN URL: https://developer.mozilla.org/ru/docs/Learn_web_development/Getting_started/Your_first_website/Styling_the_content (дата обращения: 23.11.2025).
10. HTML // MDN URL: <https://developer.mozilla.org/ru/docs/Web/HTML> (дата обращения: 23.11.2025).

Приложение 1

<https://a-surkov-2004.github.io/newRep/>

